

ITMA II

INFORME DE PROYECTO

CRUD con Laravel y MongoDB

Registro de nombres paso a paso

Crear / Consultar / Actualizar / Eliminar

ALUMNO

César Meza Corella

DOCENTE

Aquino Segura Roldán

23 de septiembre de 2026

Código fuente y guía de implementación

Objetivo y organización

Una aplicación web que persiste sus registros en MongoDB

El proyecto permite administrar nombres desde el navegador mediante cuatro operaciones: crear, consultar, actualizar y eliminar. Laravel recibe las solicitudes y valida los datos; MongoDB guarda los documentos de la colección nombres.

Se completó la conexión real a MongoDB, se ajustaron las pruebas y se verificó el recorrido en el navegador. Este informe explica la instalación, los archivos principales y las comprobaciones realizadas.

Herramientas utilizadas

- PHP 8.5.4 con la extensión mongodb habilitada.
- Laravel 13.33.0 y Composer para administrar dependencias.
- mongodb/laravel-mongodb 5.11.0 y mongodb/mongodb 2.4.2.
- MongoDB 7.0.43 en Docker y PHPUnit 12.5.35.

Cómo se organiza el proyecto

routes/web.php define las direcciones. NombreController.php coordina las acciones. Nombre.php representa los documentos. Las vistas de resources/views presentan los formularios y resultados.

Recorrido de una petición: navegador → ruta → controlador → modelo → MongoDB. Después, Laravel devuelve una vista HTML o una redirección.

Contenido de la guía

- 3 · Instalación y conexión
- 4 · Modelo y rutas
- 5 · Controlador y validaciones
- 6 · Crear y consultar
- 7 · Editar y eliminar
- 8 · Pruebas, publicación y referencias

Instalación y conexión

Pasos 1 a 3 para ejecutar una copia del repositorio

1 Descargar el código e instalar dependencias

```
git clone https://github.com/YvnPretty/cru-laravelpapuxd.git
cd cru-laravelpapuxd
composer install
cp .env.example .env
php artisan key:generate
```

Comprueba la extensión con `php --ri mongodb`. Durante la implementación se agregó la integración con `composer require mongodb/laravel-mongodb`; al clonar, `composer install` usa las versiones del archivo `composer.lock`.

2 Iniciar el servidor de MongoDB

```
docker run -d --name mongo-crud \
-p 127.0.0.1:27017:27017 \
-v mongo_crud_data:/data/db mongo:7
```

Este comando es para una instalación nueva. Si ya existe el contenedor, usa `docker start mongo-crud`. En el equipo de la práctica se reutilizó `mongo-laravel7`; no se creó otro servidor en el mismo puerto.

3 Configurar el entorno y arrancar Laravel

```
DB_CONNECTION=mongodb
DB_URI=mongodb://127.0.0.1:27017
DB_DATABASE=cru_nombres
SESSION_DRIVER=file
CACHE_STORE=file
QUEUE_CONNECTION=sync
```

```
php artisan config:clear
php artisan serve --host=127.0.0.1 --port=8000
```

Abre la dirección local que muestra Artisan. Este CRUD crea la colección al guardar el primer documento; no necesita ejecutar las migraciones SQL originales. Sus vistas usan estilos integrados y no requieren compilar Vite para esta demostración.

Modelo y rutas

Pasos 4 y 5 para enlazar Laravel con los documentos

4 Registrar la conexión y definir el modelo

En config/database.php, dentro de connections, se agregó la conexión siguiente. La opción default toma DB_CONNECTION y utiliza mongodb como valor predeterminado.

```
'mongodb' => [
    'driver' => 'mongodb',
    'dsn' => env('DB_URI', 'mongodb://127.0.0.1:27017'),
    'database' => env('DB_DATABASE', 'crud_nombres'),
],
```

El modelo app/Models/Nombre.php hereda del modelo del paquete MongoDB, selecciona esa conexión y utiliza la colección nombres.

```
namespace App\Models;

use MongoDB\Laravel\Eloquent\Model;

class Nombre extends Model
{
    protected $connection = 'mongodb';
    protected $table = 'nombres';
}
```

Cada documento contiene nombre, el identificador _id y las marcas created_at y updated_at. La integración permite acceder al identificador con id y usarlo en los enlaces de las vistas.

5 Definir las rutas del CRUD

GET	/nombres	index
GET	/nombres/create	create
POST	/nombres	store
GET	/nombres/{nombre}	show
GET	/nombres/{nombre}/edit	edit
PUT	/nombres/{nombre}	update
DELETE	/nombres/{nombre}	destroy

Las rutas están declaradas en routes/web.php. La raíz también abre el listado. Laravel resuelve el registro recibido por la ruta para los parámetros Nombre del controlador; un registro inexistente devuelve 404.

Controlador y validaciones

Paso 6 para procesar los datos del formulario

NombreController separa las acciones. index obtiene los nombres con latest()->get(); create muestra el formulario; show y edit reciben el documento correspondiente.

Crear un documento

```
$data = $request->validate([
    'nombre' => ['required', 'string', 'max:255'],
]);
$nombre = new Nombre;
$nombre->nombre = $data['nombre'];
$nombre->save();

return redirect()->route('nombres.show', $nombre);
```

El nombre es obligatorio, debe ser texto y admite hasta 255 caracteres. Solo el campo validado se asigna al modelo. Si la validación falla, Laravel regresa al formulario con los errores y conserva los datos anteriores.

Actualizar y eliminar

update aplica las mismas reglas, asigna el nuevo nombre y ejecuta save(). Después redirige a la consulta del registro. destroy ejecuta delete() y vuelve al listado.

```
$nombre->delete();

return redirect()->route('nombres.index');
```

Formularios Blade

Los formularios incluyen @csrf. La edición incorpora @method('PUT') y la eliminación @method('DELETE'), porque los formularios HTML envían POST. Los mensajes se muestran con @error y los valores previos con old().

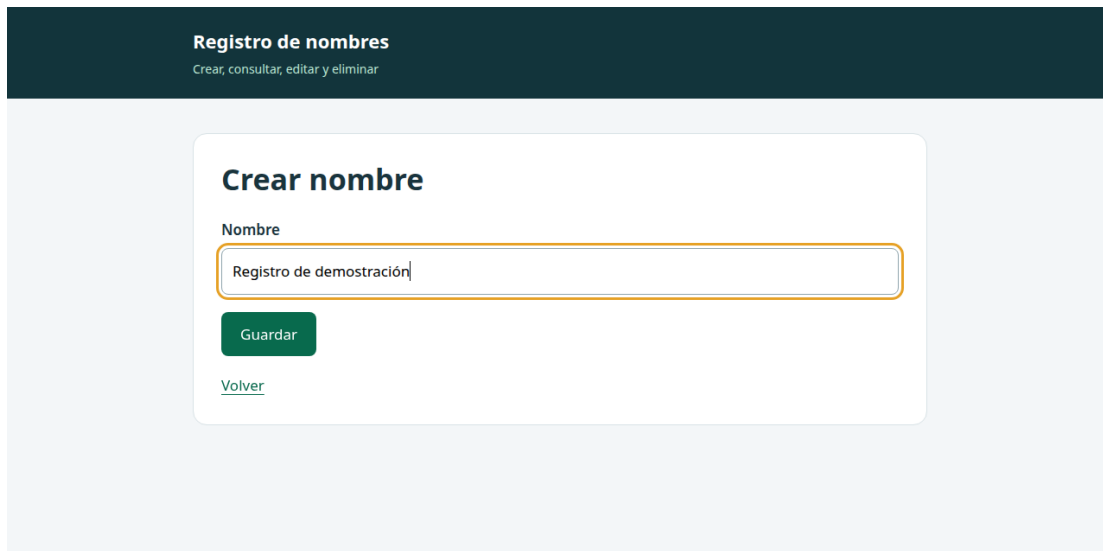
Las vistas imprimen nombres con {{ ... }}, que escapa HTML. Así, un texto que contenga etiquetas se presenta como texto y no se ejecuta como código en el navegador.

La interfaz comparte el diseño de layouts/main.blade.php: encabezado, tarjeta de contenido, campos legibles y adaptación a pantallas pequeñas. Es una práctica local sin inicio de sesión ni permisos por usuario.

Crear y consultar

Paso 7 con capturas reales de la aplicación

Desde el listado se selecciona Crear nombre, se escribe el dato y se presiona Guardar. El formulario envía la solicitud POST al controlador.



The screenshot shows a web application interface. At the top, there is a dark blue header with the text "Registro de nombres" and a subtitle "Crear, consultar, editar y eliminar". Below the header, the main content area is light gray. In the center, there is a white rounded rectangle with the title "Crear nombre". Inside this rectangle, there is a label "Nombre" above a text input field. The input field contains the text "Registro de demostración". Below the input field, there is a green button labeled "Guardar". At the bottom left of the white rectangle, there is a link labeled "Volver".

Figura 1. Formulario con el dato Registro de demostración antes de guardarlo.

Al guardar, la aplicación redirige al detalle. La vista permite abrir la edición, regresar al listado o eliminar el documento.



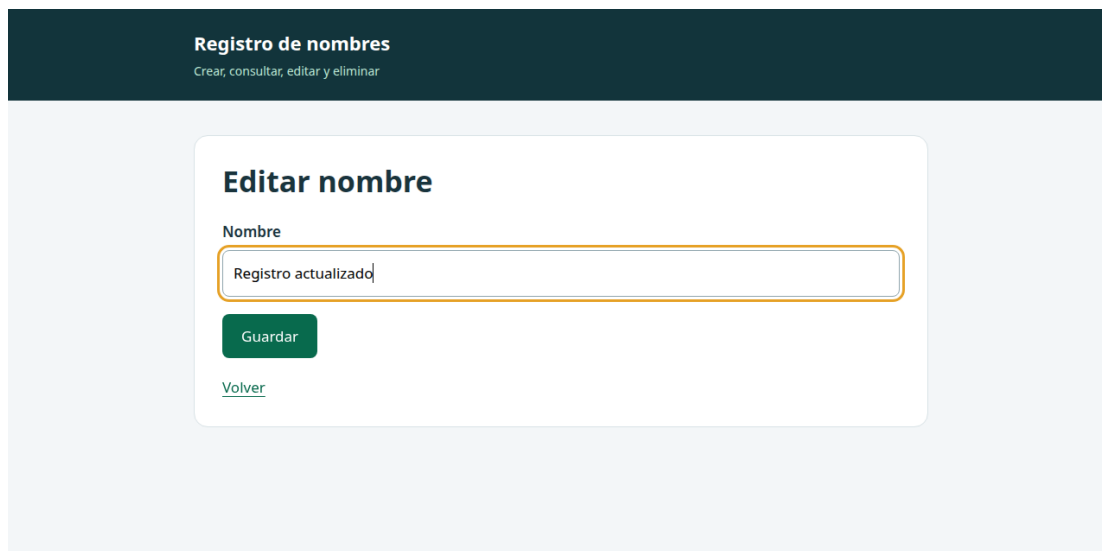
The screenshot shows the same web application interface as Figure 1. The header and main content area are the same. In the center, there is a white rounded rectangle with the title "Registro de demostración". Below the title, there are two links: "Editar" and "Volver". Below these links, there is a green button labeled "Eliminar".

Figura 2. Consulta del documento después de su creación en MongoDB.

Editar y eliminar

Paso 8 para completar el recorrido del CRUD

En el detalle se selecciona Editar. El formulario carga el nombre guardado; se cambia a Registro actualizado y se guarda. La consulta mostró correctamente el nuevo valor.



Registro de nombres
Crear, consultar, editar y eliminar

Editar nombre

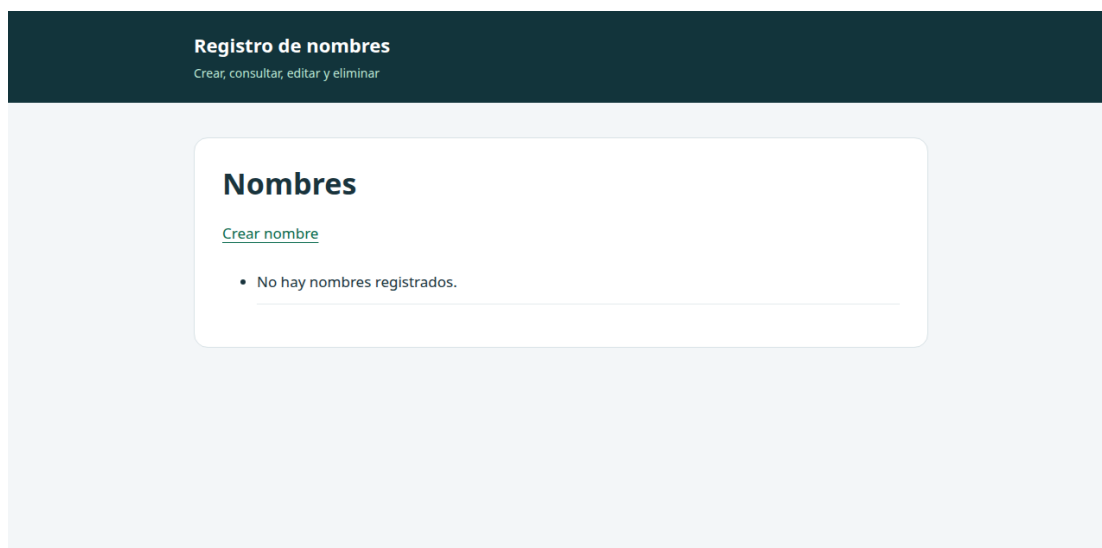
Nombre

[Guardar](#)

[Volver](#)

Figura 3. Edición del registro antes de enviar la actualización.

Desde la consulta se presiona Eliminar. El controlador elimina el documento y devuelve el listado. Al terminar la demostración no quedó el registro de prueba.



Registro de nombres
Crear, consultar, editar y eliminar

Nombres

[Crear nombre](#)

- No hay nombres registrados.

Figura 4. Listado vacío después de eliminar el documento de demostración.

Verificación y código fuente

Pasos 9 y 10 para comprobar y compartir el trabajo

9 Ejecutar las pruebas

```
php artisan test --compact  
vendor/bin/pint --dirty --format agent  
composer validate --no-check-publish
```

Resultado obtenido: 5 pruebas aprobadas y 46 aserciones. Se comprobó el ciclo CRUD, la persistencia de los cambios, el rechazo de nombres inválidos, el escape de HTML y la respuesta de la página principal. Pint y la validación de Composer finalizaron correctamente.

Las pruebas utilizan una base temporal con el prefijo `crud_nombres_test_` y un identificador único. Al terminar se elimina únicamente esa base. No usan SQLite como sustituto de MongoDB ni ejecutan las migraciones SQL del proyecto original.

10 Publicar el proyecto en GitHub

Se inicializó Git, se revisaron las exclusiones de `.gitignore`, se creó un commit y se envió la rama `main`. Los cambios finales incluyen la integración de MongoDB y este informe. El token de acceso no forma parte del código ni del documento.

```
git add .  
git commit -m "Complete Laravel MongoDB CRUD and report"  
git push -u origin main
```

Repositorio del proyecto

<https://github.com/YvnPretty/cru-laravelpapuxd>

Se versionan el código, `composer.lock` y `.env.example`. Se excluyen `.env`, `vendor`, `node_modules` y los registros. La guía `README.md` del repositorio incluye las instrucciones para instalar y probar la aplicación.

Resultado y referencias

El proyecto permite realizar las cuatro operaciones con MongoDB desde una interfaz web. La evidencia combina pruebas automáticas y un recorrido real por los formularios.

MongoDB · Guía oficial de conexión con Laravel

MongoDB · Modelos Eloquent

Consulta de documentación: 23 de septiembre de 2026.